Simulación

Eje Temático Nº 4

Lenguajes y Software
de Simulación

4to Nivel
Ingeniería en Sistemas de Información

Ing. Revilla, Sergio Andrés
Ing. Sofía Gaggi

Contenidos

1. Traslación del modelo a la computadora 3
¿En qué parte del proceso estamos? 3
Qué significa «trasladar» un modelo 3
Los componentes de un modelo ejecutable 4
Un ejemplo que nos va a acompañar todo el eje 5
2. El motor de simulación: cómo avanza el tiempo 6
Dos maneras de mover el reloj 6
El ciclo de ejecución 7
3. Lenguajes de simulación 8
¿Por qué hicieron falta lenguajes propios? 8
Las dos visiones del mundo 9
Visión orientada a eventos 9
Visión orientada a procesos 10
Las dos visiones, comparadas 11
Una tercera vuelta de tuerca: la orientación a objetos 11
Los lenguajes clásicos y qué aportó cada uno 12
Los «lenguajes» de hoy: bibliotecas sobre lenguajes de propósito general 12
4. Software de simulación 13
Cuatro familias de herramientas 13
No todo el software simula lo mismo 14
Panorama de herramientas 15
Qué resuelve el software por nosotros y qué no 16
5. Implementación y verificación del modelo 17

Cómo se construye el modelo en un simulador 17

Verificación: ¿está bien construido el modelo? 17

Verificar no es validar 18

Semillas, réplicas y reproducibilidad 19

Complemento A — Evolución de los lenguajes y el software 20

Complemento B — Criterios para seleccionar una herramienta 21

Referencias bibliográficas 22

Bibliografía obligatoria de la cátedra 22

Bibliografía complementaria 22

Documentación de herramientas 22

1. TRASLACIÓN DEL MODELO A LA COMPUTADORA

¿EN QUÉ PARTE DEL PROCESO ESTAMOS?

En el Eje 1 vimos que un estudio de simulación no es «sentarse a programar», sino un proceso de diez etapas que arranca en la definición del problema y termina en la documentación de los resultados. La cuarta de esas etapas —**Desarrollo del Modelo de Simulación**— consiste, textualmente, en «traducir el modelo conceptual en un modelo de simulación computarizado». De eso trata este eje.

Hasta acá venimos construyendo las piezas sueltas. En el **Eje 2** conseguimos una fuente de aleatoriedad: los generadores congruenciales que producen números pseudoaleatorios uniformes en [0,1), reproducibles y sometidos a pruebas estadísticas. En el **Eje 3** aprendimos a transformar esos uniformes en la distribución que necesitemos —Transformada Inversa, Rechazo, Composición— y usamos esos valores en simulaciones de Monte Carlo, que son **estáticas**: no tienen tiempo.

Lo que falta es justamente el tiempo. Un modelo de una línea de producción, de una guardia hospitalaria o de un centro de distribución necesita que las cosas ocurran **en un orden y en un momento**: primero llega el cliente, después espera, después lo atienden, después se va. Ese mecanismo —el que hace avanzar el reloj y ordena los sucesos— es el corazón de un simulador y es lo primero que vamos a estudiar en este eje temático.

> **Idea clave:** el Eje 2 nos dio el motor de aleatoriedad y el Eje 3 las máquinas de transformación. El Eje 4 agrega la tercera pieza: **el motor de tiempo**. Con las tres juntas ya se puede simular un sistema dinámico completo.

QUÉ SIGNIFICA «TRASLADAR» UN MODELO

Trasladar un modelo a la computadora significa **reescribir la descripción del sistema en términos de estructuras de datos y reglas ejecutables**. El modelo conceptual dice, en castellano, algo así como «los clientes llegan cada tanto, hacen fila y son atendidos por un cajero». El modelo computacional tiene que decir eso mismo pero en términos de listas, variables, contadores y procedimientos que una máquina pueda ejecutar millones de veces.

Esa traducción tiene un costo: **todo lo que no se traduce, no existe en el modelo**. Si el modelo conceptual mencionaba que el cajero se toma un descanso a media mañana y eso no se codificó, el simulador va a producir resultados perfectamente prolijos de un sistema que no es el nuestro. Por eso, más adelante en este apunte, la verificación tiene una sección propia.

LOS COMPONENTES DE UN MODELO EJECUTABLE

Casi cualquier modelo de eventos discretos —programado a mano o construido con arrastrar y soltar— se organiza alrededor del mismo conjunto de componentes. Conviene conocerlos con nombre propio porque son el vocabulario común de todos los lenguajes y programas de simulación.



Componente	Qué es	En una caja de minimercado
Entidad	Objeto que circula por el sistema y sobre el que ocurren las cosas. Se crea, se mueve y se destruye.	Cada cliente
Atributo	Dato propio de cada entidad, que la distingue de las demás.	Hora de llegada, cantidad de artículos, prioridad
Recurso	Medio limitado que las entidades necesitan y se disputan. Puede estar libre, ocupado, en falla o fuera de turno.	El cajero
Cola	Lugar de espera de las entidades que no pueden avanzar todavía. Tiene una **disciplina** que decide a quién le toca primero.	La fila frente a la caja (FIFO)
Variable de estado	Dato global que describe la situación del sistema en un instante.	Cantidad de clientes en fila; estado del cajero

Componente	Qué es	En una caja de minimercado
Evento	Suceso instantáneo que **cambia** el estado del sistema.	Llegada de un cliente; fin de una atención
Reloj de simulación	Variable que contiene el tiempo simulado actual. No tiene nada que ver con el tiempo real de ejecución.	t = 0, 4, 5, 6, 8, … minutos
Lista de eventos futuros	Agenda ordenada por tiempo con los eventos ya programados que todavía no ocurrieron. Es la estructura central del simulador.	{ Fin de atención en t=5; Llegada en t=6 }
Acumuladores estadísticos	Contadores y sumas que se van actualizando durante la corrida para poder calcular los indicadores al final.	Suma de esperas, tiempo total ocupado del cajero
Réplica	Una corrida completa del modelo. Como el modelo es estocástico, una sola corrida no alcanza: hacen falta varias.	Simular 100 veces el mismo día de trabajo

Puente con el Eje 5: los términos **cola**, **disciplina de cola**, **tiempo de espera**, **utilización del recurso** y **longitud media de la fila** aparecen acá como piezas del modelo computacional, sin desarrollo matemático. En el Eje 5 (Simulación de Sistemas Discretos) los vamos a retomar desde la **Teoría de Colas**, con notación propia y fórmulas analíticas que permiten contrastar los resultados del simulador contra un resultado teórico.

UN EJEMPLO QUE NOS VA A ACOMPAÑAR TODO EL EJE

Para no hablar en abstracto, vamos a usar un sistema mínimo pero completo: **una única caja de un minimercado**.

- **Entidad:** el cliente, con el atributo «hora de llegada».
- **Recurso:** un cajero (capacidad 1).
- **Cola:** la fila, con disciplina FIFO y capacidad ilimitada.
- **Variables aleatorias:** el tiempo entre llegadas y el tiempo de atención. Ambas se generan con los métodos del Eje 3, a partir de los números pseudoaleatorios del Eje 2.
- **Eventos:** llegada de un cliente y fin de atención.
- **Resultados buscados:** espera promedio en fila, tiempo promedio en el sistema y porcentaje de utilización del cajero.

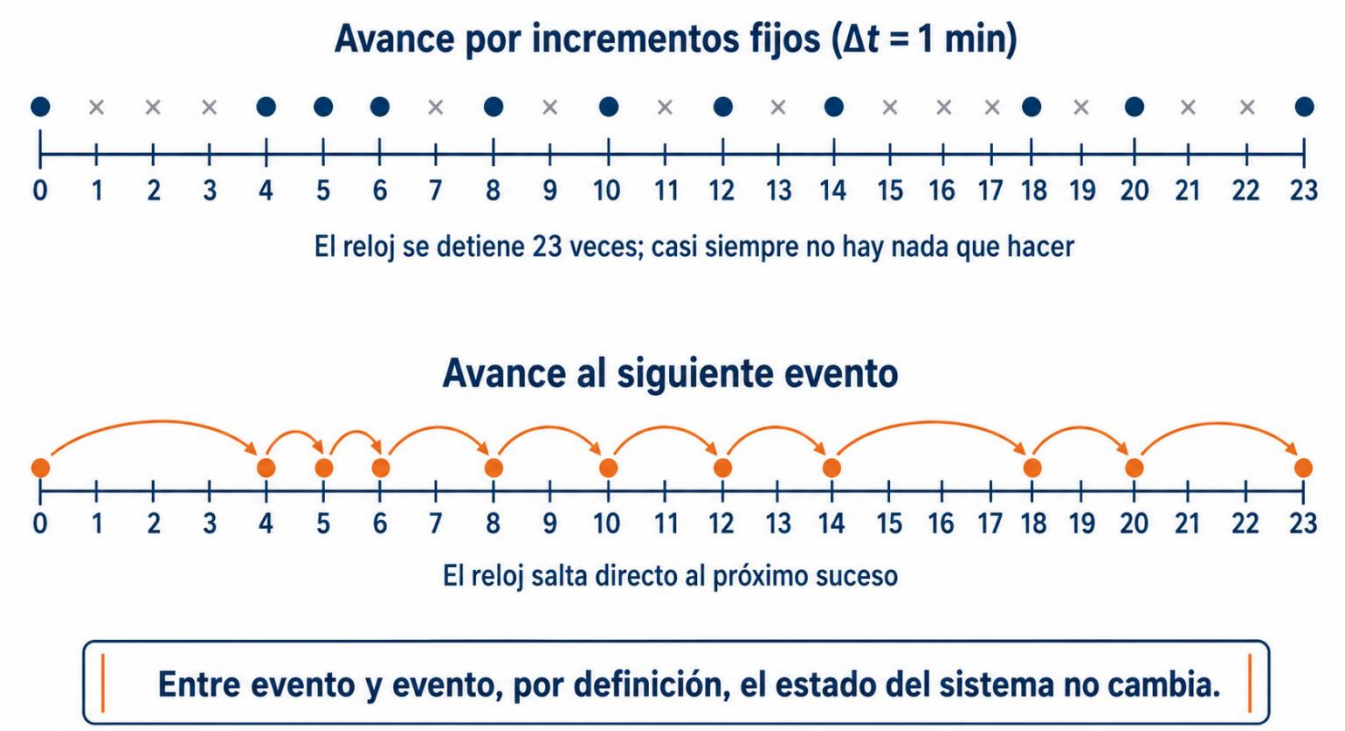
Es el mismo sistema que después vamos a expresar de tres maneras distintas —como eventos, como procesos y como objetos— y que en el laboratorio se construye en pocos minutos con un simulador visual. Simple como es, ya contiene todo lo esencial de la simulación de eventos discretos.

2. EL MOTOR DE SIMULACIÓN: CÓMO AVANZA EL TIEMPO

DOS MANERAS DE MOVER EL RELOJ

Un modelo dinámico necesita decidir **cómo avanza el reloj de simulación**. Existen dos mecanismos, y la diferencia entre ellos explica buena parte de la eficiencia de los simuladores modernos.

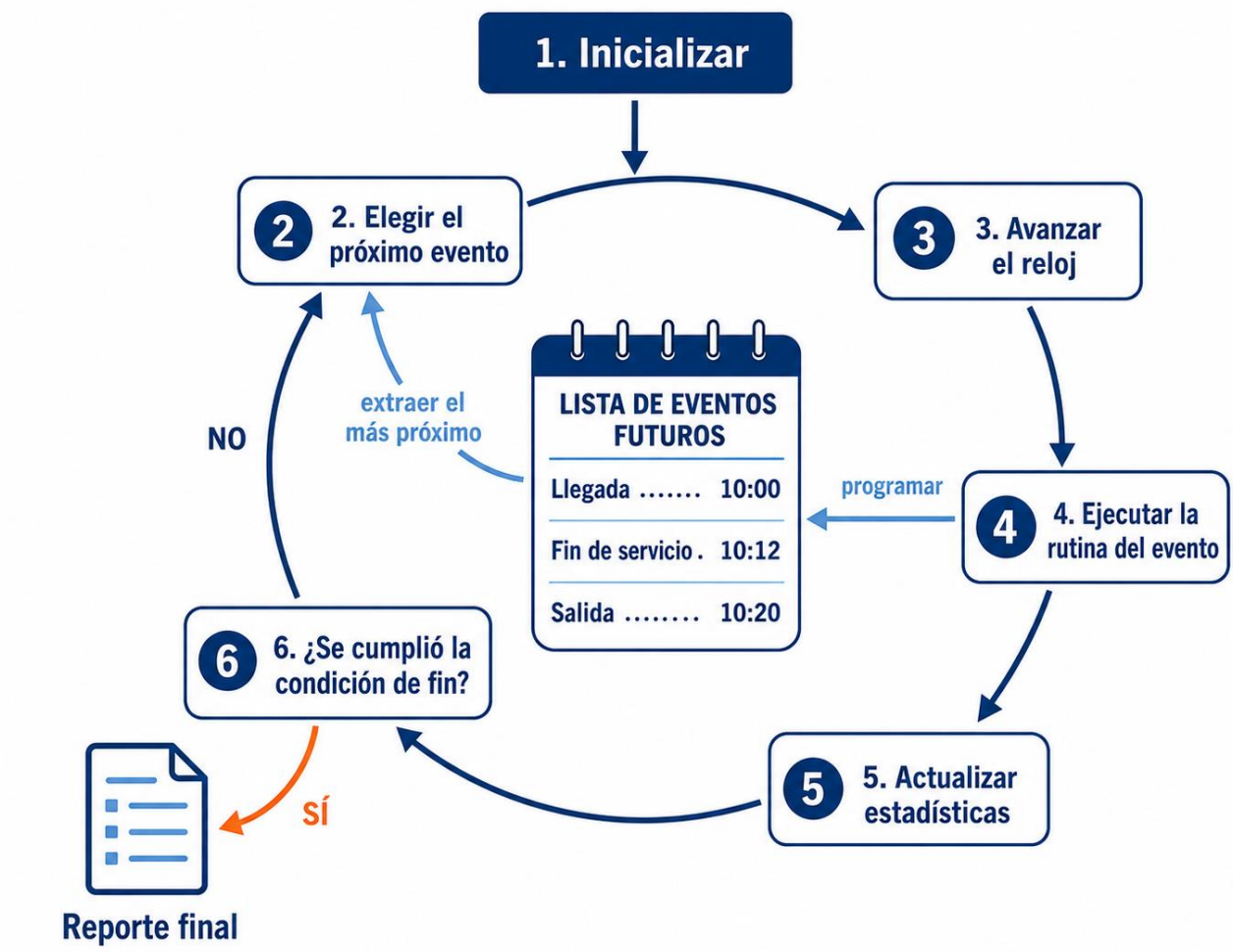
Mecanismo	Cómo funciona	Cuándo se usa
Incrementos fijos de tiempo	El reloj avanza siempre el mismo paso Δt (un segundo, un minuto, una hora). En cada paso se revisa si ocurrió algo.	Sistemas **continuos**, donde el estado cambia permanentemente y hay que integrar ecuaciones paso a paso (Eje 8).
Avance al siguiente evento	El reloj salta directamente al instante del próximo evento programado. Entre evento y evento, por definición, no cambia nada.	Simulación de **eventos discretos**. Es el mecanismo dominante en prácticamente todos los simuladores de este tipo.



La ventaja del segundo mecanismo es evidente: si en nuestra caja de minimercado solo pasan cosas cuando llega alguien o cuando termina una atención, no tiene ningún sentido recalcular el estado del sistema segundo a segundo durante ocho horas. El simulador salta de suceso en suceso y hace, literalmente, la mínima cantidad de trabajo necesaria.

Esto también explica algo que suele sorprender: **el tiempo simulado y el tiempo real de ejecución no tienen relación**. Un modelo puede simular un año de operación de una planta en cuarenta segundos de reloj de pared, o al revés, un modelo muy detallado de diez minutos puede tardar horas en correr.

EL CICLO DE EJECUCIÓN



Con el mecanismo de siguiente evento, todo simulador de eventos discretos —sin importar la marca ni el lenguaje— ejecuta el mismo ciclo:

1. **Inicialización.** Poner el reloj en cero, fijar el estado inicial del sistema, poner en cero los acumuladores estadísticos y programar los primeros eventos (típicamente, la primera llegada).
2. **Selección del próximo evento.** Buscar en la lista de eventos futuros el evento con el menor tiempo programado y sacarlo de la lista.
3. **Avance del reloj.** Llevar el reloj de simulación hasta el instante de ese evento.

4. **Ejecución de la rutina del evento.** Modificar las variables de estado, mover entidades entre colas y recursos y **programar los eventos futuros que se desprendan de este**.
5. **Actualización de estadísticas.** Acumular esperas, tiempos ocupados, cantidades atendidas.
6. **Control de fin.** Si se cumplió la condición de corte (tiempo final alcanzado, cantidad de entidades procesadas, lista de eventos vacía), terminar y emitir el reporte. Si no, volver al paso 2.

Regla de desempate: puede ocurrir que dos eventos queden programados para el **mismo instante**. El simulador necesita una regla explícita para decidir cuál procesa primero, porque el resultado puede cambiar. La convención más habitual —y la que usamos acá— es **procesar primero los fines de servicio y después las llegadas**: así el recurso queda liberado antes de que el nuevo cliente pregunte si está libre.

3. LENGUAJES DE SIMULACIÓN

¿POR QUÉ HICIERON FALTA LENGUAJES PROPIOS?

En los años cincuenta, los primeros modelos de simulación se escribían íntegramente en lenguajes de propósito general —FORTRAN, ALGOL, más tarde C—. El analista tenía que programar, además de la lógica de su sistema, **toda la infraestructura**: la lista de eventos futuros y su ordenamiento, el avance del reloj, las colas, la administración de recursos, los generadores aleatorios y la recolección de estadísticos. Buena parte del esfuerzo se iba en código que no tenía nada que ver con el problema estudiado y que se reescribía, con sus errores, en cada proyecto nuevo.

De ahí nace la idea de **lenguaje de simulación**: un lenguaje cuyas instrucciones primitivas ya son los conceptos de la simulación. En vez de escribir «insertar un registro en una lista enlazada ordenada por el campo tiempo», uno escribe «programar el evento fin de servicio dentro de 4 minutos». En vez de manejar índices, uno declara una cola y un recurso.

Definición: un **lenguaje de simulación** es un lenguaje de programación cuyas construcciones básicas —tipos de datos, sentencias, estructuras de control— corresponden directamente a los conceptos del modelado de simulación: entidades, atributos, recursos, colas, eventos, procesos y avance del reloj. El motor de simulación viene incorporado en el lenguaje, no lo escribe el usuario.

La consecuencia práctica es enorme: el modelo se escribe en menos líneas, se lee mejor y contiene muchos menos errores, porque la parte más delicada —la agenda de eventos— ya viene probada. El precio es cierta pérdida de libertad frente a un lenguaje de propósito general.

LAS DOS VISIONES DEL MUNDO

El punto realmente conceptual de este eje es el siguiente: **el mismo sistema puede describirse de dos maneras distintas**, y esas dos maneras dieron origen a dos familias de lenguajes de simulación. En la literatura se las llama «visiones del mundo» (world views).



VISIÓN ORIENTADA A EVENTOS

El modelo se organiza alrededor de los **eventos**. Para cada tipo de evento se escribe una rutina que dice qué cambia cuando ese evento ocurre y qué eventos nuevos se programan. Es exactamente lo que hicimos a mano en la traza de la sección anterior.

En nuestro minimercado hay dos rutinas de evento:

```
rutina LLEGADA(cliente):
    programar LLEGADA del próximo cliente en
        reloj + generar_tiempo_entre_llegadas()
    si cajero = LIBRE:
        cajero ← OCUPADO
        acumular espera 0
        programar FIN_ATENCION en reloj + generar_tiempo_atencion()
    sino:
        encolar(cliente, hora_llegada = reloj)
```

```
rutina FIN_ATENCION(cliente):
    registrar salida del cliente y su tiempo en el sistema
    si fila está vacía:
        cajero ← LIBRE
    sino:
        c ← desencolar()
        acumular espera (reloj – c.hora_llegada)
        programar FIN_ATENCION en reloj + generar_tiempo_atencion()
```

Fijate que el cliente **no aparece por ningún lado como protagonista**. El modelo es una colección de reacciones del sistema ante sucesos. Esta visión obliga a pensar con mucha precisión qué cambia, cuándo y con qué consecuencias, y por eso es insuperable para entender cómo funciona un simulador por dentro. Su desventaja aparece en modelos grandes: con veinte tipos de eventos entrelazados, seguir el hilo se vuelve difícil y el código se aleja bastante de cómo uno describiría el sistema hablando.

VISIÓN ORIENTADA A PROCESOS

Acá el modelo se organiza alrededor del **ciclo de vida de las entidades**. Se escribe, de corrido, la historia que vive un cliente desde que entra hasta que se va:

```
proceso CLIENTE:
    hora_llegada ← reloj
    tomar(cajero)                    # si está ocupado, el proceso
                                     # se suspende acá hasta que se libere
    acumular espera (reloj – hora_llegada)
    demorar(generar_tiempo_atencion())
    liberar(cajero)
    registrar tiempo en el sistema

proceso GENERADOR_DE_CLIENTES:
    repetir:
        demorar(generar_tiempo_entre_llegadas())
        crear una nueva instancia del proceso CLIENTE
```

La diferencia es notable. Este texto **se lee como el enunciado del problema**: el cliente llega, toma el cajero, espera si está ocupado, se hace atender, lo libera y se va. No hay ninguna mención al reloj, a la lista de eventos ni a la fila: la fila aparece sola, como consecuencia de que varios procesos quedaron suspendidos en la instrucción «tomar(cajero)».

Las dos instrucciones clave son las que **suspenden** el proceso: «demorar()», que lo deja dormido una cantidad conocida de tiempo simulado, y «tomar()», que lo deja dormido por un tiempo desconocido hasta que el recurso

esté disponible. Todo lo que hace un lenguaje orientado a procesos es administrar esas suspensiones y reanudaciones.

LAS DOS VISIONES, COMPARADAS

	Orientación a eventos	Orientación a procesos
El modelo se escribe como…	un conjunto de rutinas, una por cada tipo de suceso	una narración del recorrido de cada entidad
El protagonista es…	el sistema, que reacciona	la entidad, que actúa
Reloj y agenda	explícitos: los programa el modelador	implícitos: los resuelve el lenguaje
Fortaleza	control total y máxima eficiencia	legibilidad y rapidez de construcción
Debilidad	se vuelve difícil de seguir en modelos grandes	menos control fino sobre situaciones muy particulares
Ejemplos	SIMSCRIPT; modelos escritos a mano en C, Java o Python	GPSS, SIMULA, SimPy, y casi todos los simuladores visuales

Lo más importante de esta sección: las dos visiones **no compiten**: describen el mismo sistema desde distinta altura. Y hay un detalle decisivo: por debajo, **un lenguaje orientado a procesos se ejecuta con un motor orientado a eventos**. Cuando un proceso ejecuta «demorar(4)», el motor no hace otra cosa que programar en la agenda un evento «reanudar este proceso en reloj + 4». La orientación a procesos no es un motor distinto: es una **interfaz más cómoda** sobre el mismo motor.

UNA TERCERA VUELTA DE TUERCA: LA ORIENTACIÓN A OBJETOS

La tercera visión no reemplaza a las anteriores, las envuelve. En la **orientación a objetos**, cada componente del sistema —una máquina, un operario, un autoelevador, una cinta transportadora, un depósito— es un objeto que reúne sus datos, su estado y su comportamiento, y que sabe comunicarse con los demás.

Lo interesante es que esto no vino de afuera de la simulación: la programación orientada a objetos **nació dentro de la simulación**. El lenguaje SIMULA, creado a mediados de los años sesenta justamente para simular sistemas, introdujo los conceptos de clase, objeto y herencia, que después migraron a la informática general y llegaron a C++, Java, Python y todo lo demás.

La orientación a objetos es lo que hace posible el modo de trabajo actual: **bibliotecas de objetos listos para usar**. Cuando en un simulador visual uno arrastra un «procesador» a la escena, está instanciando un objeto que ya trae adentro su cola, su recurso, sus rutinas de evento y sus estadísticos. Modelar deja de ser escribir la lógica y pasa a ser **elegir, conectar y parametrizar** objetos, bajando muchísimo el tiempo de construcción y el riesgo de error.

LOS LENGUAJES CLÁSICOS Y QUÉ APORTÓ CADA UNO

Los lenguajes de simulación de la etapa clásica ya casi no se usan tal cual en la industria, pero conviene conocerlos porque **las ideas que introdujeron están adentro de todo lo que usamos hoy**.

Lenguaje	Origen	Visión	Aporte principal
GPSS	1961, IBM (G. Gordon)	Procesos (transacciones)	Modelado por **bloques**: la entidad recorre un diagrama de bloques que representa el proceso. Es el antepasado directo de la interfaz de casi todos los simuladores visuales actuales.
SIMSCRIPT	1962-63, RAND	Eventos	Descripción declarativa de entidades, atributos y conjuntos, con rutinas de evento en un lenguaje de sintaxis cercana al inglés.
SIMULA	1962-67, Centro de Cómputos de Noruega	Procesos	Introdujo **clase, objeto y herencia**. Origen histórico de la programación orientada a objetos.
GASP / SLAM / SIMAN	1969-1982, EE.UU.	Híbrida	Bibliotecas y lenguajes que combinaron eventos, procesos y variables continuas. SIMAN es el motor sobre el que se construyó Arena.
DEVS	1976 (B. Zeigler)	Formalismo	No es un lenguaje sino una **especificación matemática** de los sistemas de eventos discretos, independiente de la implementación. Se usa como base formal de varias herramientas.
Modelica	1997, consorcio internacional	Ecuaciones	Estándar abierto para modelar sistemas **continuos e híbridos** mediante ecuaciones, sin fijar de antemano qué variable se despeja.

LOS «LENGUAJES» DE HOY: BIBLIOTECAS SOBRE LENGUAJES DE PROPÓSITO GENERAL

La tendencia actual invirtió el planteo. En lugar de crear un lenguaje nuevo, se toma un lenguaje de propósito general muy difundido y se le agrega una **biblioteca** que aporta el motor de simulación. Se conserva así todo el ecosistema del lenguaje anfitrión —depuradores, control de versiones, conexión a bases de datos, bibliotecas de análisis de datos y de gráficos— y se gana el motor sin escribirlo.

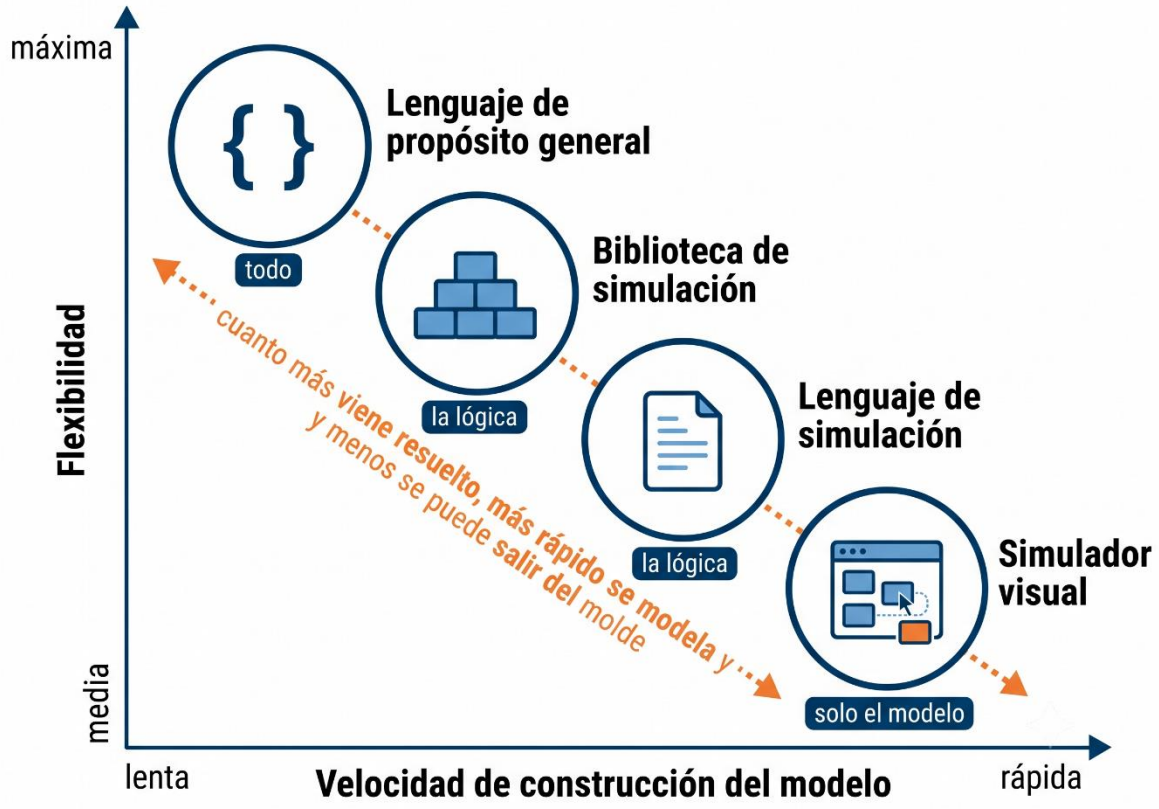
- **SimPy** (Python) es el ejemplo más claro y el más cercano a esta carrera: implementa la visión orientada a procesos usando generadores del lenguaje, con recursos compartidos, colas y eventos. El pseudocódigo de proceso que vimos más arriba es casi literalmente un modelo de SimPy.
- **simmer** (R) ofrece un enfoque similar, orientado a procesos, integrado con las herramientas estadísticas de R.
- **SSJ** (Java) y bibliotecas equivalentes en C++ y Julia cubren el mismo espacio para otros lenguajes.

Esta familia rescata algo que el alumno de Ingeniería en Sistemas debería tener presente: **saber programar sigue siendo una ventaja competitiva en simulación**. Los simuladores visuales resuelven el 90 % de los casos, pero el 10 % restante —una regla de despacho poco convencional, una integración con el sistema de gestión de la empresa, un experimento automatizado sobre miles de escenarios— se resuelve programando.

4. SOFTWARE DE SIMULACIÓN

CUATRO FAMILIAS DE HERRAMIENTAS

Con todo lo anterior en la mano, ya podemos ordenar el panorama. A la hora de implementar un modelo hay cuatro caminos posibles, que se diferencian por cuánto trabajo hace el analista y cuánto viene resuelto.

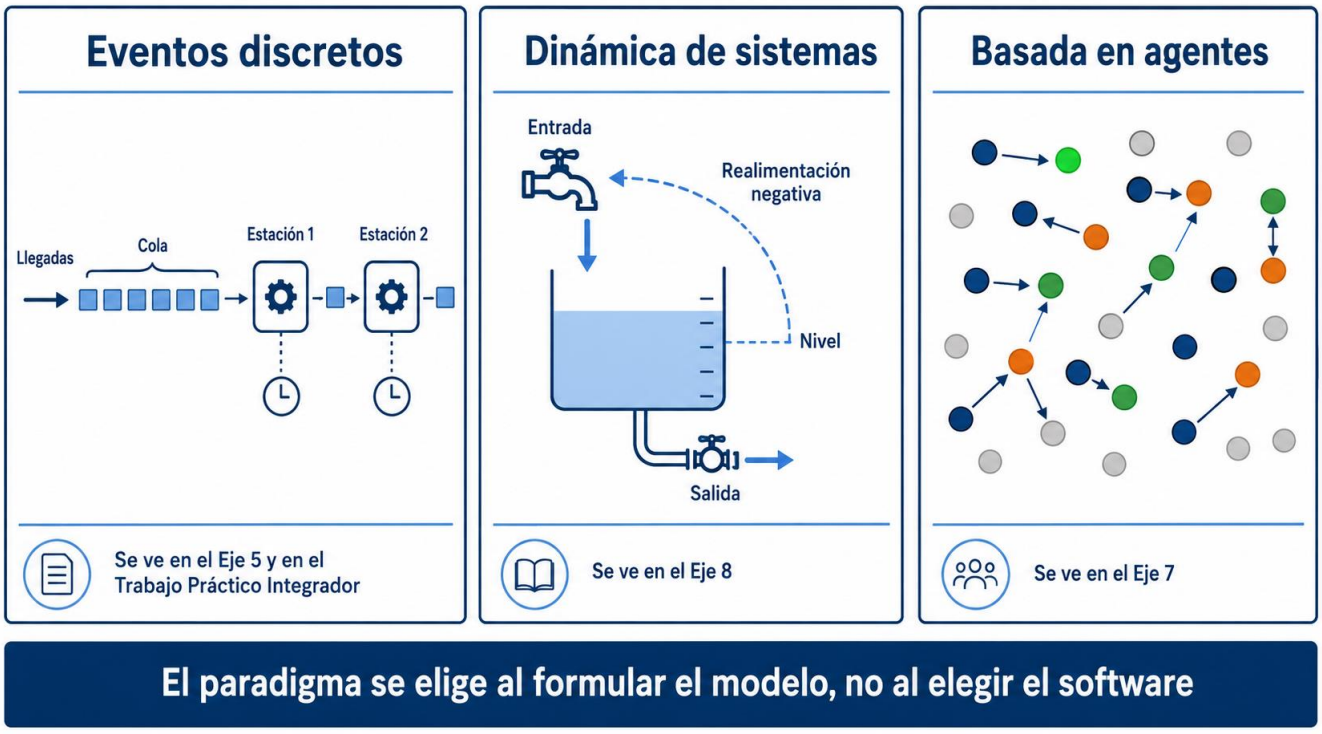


Familia	Qué aporta	Qué hay que construir	Flexibilidad	Tiempo de desarrollo
Lenguaje de propósito general (Python, Java, C++, C#)	Nada específico de simulación	Todo: agenda, reloj, colas, recursos, estadísticos	Máxima	Muy alto
Biblioteca de simulación (SimPy, simmer, SSJ)	El motor de simulación como biblioteca del lenguaje	La lógica del sistema, en código	Alta	Alto
Lenguaje de simulación (GPSS, SIMSCRIPT)	Un lenguaje cuyas primitivas son conceptos de simulación	La lógica del sistema, en ese lenguaje	Media-alta	Medio
Simulador visual (paquetes con interfaz gráfica)	Motor, objetos predefinidos, animación, estadísticos y reportes	El modelo, arrastrando y parametrizando objetos	Media	Bajo

No existe una opción universalmente mejor. La regla práctica es que **la flexibilidad y la velocidad de desarrollo son inversas**: cuanto más resuelto viene, más rápido se modela y menos se puede salir del molde. Y hay un matiz importante: los simuladores visuales modernos incorporan lenguajes de programación internos para escribir lógica personalizada, con lo cual la frontera entre la tercera y la cuarta familia es hoy bastante porosa.

NO TODO EL SOFTWARE SIMULA LO MISMO

Hasta acá hablamos de eventos discretos, pero ese es solo uno de los paradigmas de simulación, y **cada paradigma tiene su propio software**. Es un error frecuente pedirle a una herramienta lo que su paradigma no puede representar.



Paradigma	Cómo representa el sistema	Preguntas que responde	Dónde se ve en la materia
Eventos discretos (DES)	Entidades individuales que recorren procesos y compiten por recursos. El estado cambia en instantes puntuales.	¿Cuánto se espera? ¿Cuál es el cuello de botella? ¿Cuánta capacidad hace falta?	Eje 5 y Trabajo Práctico Integrador
Dinámica de sistemas (simulación continua)	Niveles y flujos ligados por bucles de realimentación, mediante ecuaciones que se integran paso a paso. No hay individuos.	¿Cómo evoluciona una población, un stock, un mercado? ¿Qué bucles dominan el comportamiento?	Eje 8 (diagramas causales y de Forrester)
Basada en agentes (ABM)	Individuos autónomos con reglas de comportamiento propias. El comportamiento global **emerge** de sus interacciones.	¿Cómo se difunde una conducta, una epidemia, una tecnología? ¿Qué pasa si los individuos son heterogéneos?	Eje 7
Multimétodo o híbrido	Combina dos o tres de los anteriores en un mismo modelo.	Problemas que tienen a la vez procesos operativos y comportamiento individual o agregado.	Mención; se trabaja en asignaturas superiores

Advertencia: elegir el paradigma es una decisión **de modelado**, no de software: se toma en la etapa de formulación del modelo conceptual (Eje 1), mucho antes de abrir un programa. Cuando el paradigma elegido no se corresponde con la naturaleza del problema, ninguna herramienta lo arregla.

PANORAMA DE HERRAMIENTAS

La lista que sigue no pretende ser exhaustiva ni un ranking; es un mapa para ubicarse. Los precios, las prestaciones y hasta los nombres cambian con los años: lo que no cambia son las categorías.

Herramienta	Tipo	Paradigma
Arena	Simulador visual por diagrama de bloques	Eventos discretos
FlexSim	Simulador visual 3D orientado a objetos	Eventos discretos (con módulo de agentes)
Simio	Simulador visual orientado a objetos	Eventos discretos y agentes
AnyLogic	Plataforma multimétodo	Discreto, continuo y agentes
Simul8, Witness, ExtendSim, ProModel, Plant Simulation	Simuladores visuales de propósito general o industrial	Eventos discretos
JaamSim	Simulador visual libre y de código abierto	Eventos discretos
SimPy (Python), simmer (R)	Bibliotecas sobre lenguajes de propósito general	Eventos discretos, orientado a procesos
GPSS World, SIMSCRIPT III	Lenguajes de simulación	Procesos / eventos

Herramienta	Tipo	Paradigma
Vensim, Stella, Powersim	Simuladores de **dinámica de sistemas**	Continuo
NetLogo, Repast, Mesa	Plataformas de modelado de agentes	Basada en agentes
OMNeT++, ns-3	Simuladores de dominio específico (redes)	Eventos discretos

En el laboratorio de la cátedra se trabaja con **FlexSim**, un simulador visual 3D orientado a objetos que se usa en el Trabajo Práctico Integrador para la parte de eventos discretos, y con **Vensim**, un simulador de dinámica de sistemas que se emplea en el Eje 8 para los modelos continuos. Son dos herramientas de paradigmas distintos, y esa es precisamente la razón por la que se usan las dos.

QUÉ RESUELVE EL SOFTWARE POR NOSOTROS Y QUÉ NO

Es útil tener claro, con nombre y apellido, qué parte del trabajo automatiza una herramienta de simulación:

Lo que el software resuelve	Lo que sigue siendo nuestro
La agenda de eventos futuros y el avance del reloj.	La **formulación del problema** y la definición de qué se quiere responder.
Los **generadores de números pseudoaleatorios** con período largo y probados (Eje 2).	La **recolección y el análisis de los datos de entrada**: qué distribución representa realmente a las llegadas o a los tiempos de servicio (Eje 6).
Los **métodos de generación de variables aleatorias** de todas las distribuciones usuales (Eje 3).	El **modelo conceptual**: qué se incluye, qué se deja afuera y con qué nivel de detalle.
La administración de colas, recursos, disciplinas y prioridades.	La **verificación** de que el modelo construido es el modelo pensado.
La recolección de estadísticos, la animación y los reportes.	La **validación** contra el sistema real y la **interpretación** de los resultados.
La ejecución de réplicas, escenarios y experimentos.	El **diseño** de esos experimentos y las conclusiones que se extraen.

> Todo lo que estudiamos en los Ejes 2 y 3 está adentro de cualquier simulador. Cuando en una herramienta se configura una llegada exponencial(5) o un tiempo de proceso normal(10, 2), lo que se está invocando es un generador congruencial seguido de un método de transformación. **No lo estudiamos para programarlo, sino para saber qué está haciendo la herramienta y poder juzgar si está bien usada.**

5. IMPLEMENTACIÓN Y VERIFICACIÓN DEL MODELO

CÓMO SE CONSTRUYE EL MODELO EN UN SIMULADOR

En un simulador visual, la implementación consiste en llevar cada componente del modelo conceptual a un objeto de la herramienta. Los nombres cambian de un programa a otro, pero la correspondencia es siempre la misma:

Componente del modelo	Objeto típico en el simulador
Generación de entidades según una distribución de tiempo entre llegadas	Fuente / source / bloque de creación
Cola de espera con su disciplina y capacidad	Cola / queue / buffer
Recurso que procesa durante un tiempo aleatorio	Procesador / servidor / máquina
Traslado entre estaciones	Cinta, camino, transporte, operario
Salida del sistema y conteo de entidades procesadas	Sumidero / sink
Lógica que no encaja en los objetos estándar	Editor de procesos o código interno del simulador

Una vez armada la estructura, resta parametrizar: distribuciones de cada tiempo, capacidades, disciplinas de cola, turnos, fallas, reglas de decisión. Recién entonces el modelo puede correrse, y ahí empieza la parte que más se subestima.

VERIFICACIÓN: ¿ESTÁ BIEN CONSTRUIDO EL MODELO?

Verificar es comprobar que el modelo implementado hace lo que nosotros quisimos que hiciera; es decir, que la traducción del modelo conceptual al modelo computacional fue correcta. Es una tarea puramente interna, y ninguna herramienta la hace sola. Algunas técnicas concretas:

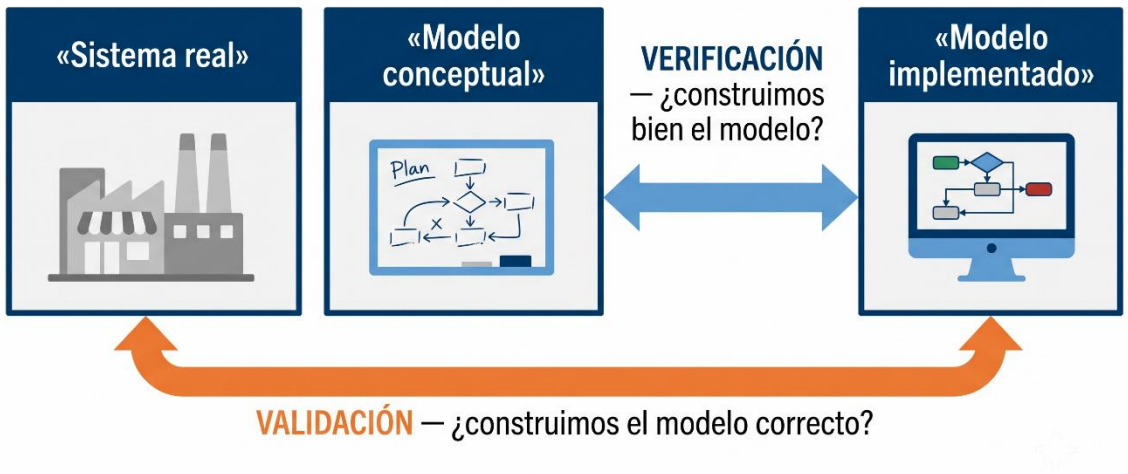
- **Correr con valores determinísticos.** Reemplazar todas las distribuciones por constantes: llegadas cada 5 minutos exactos y atención de 3 minutos exactos. El resultado es predecible a mano y cualquier desvío delata un error.
- **Revisar la traza de eventos.** Casi todos los simuladores permiten listar los eventos en orden y seguir una entidad en particular. Es la versión automática de la tabla que hicimos en la sección 2.
- **Mirar la animación.** Sirve para detectar disparates evidentes: entidades que se acumulan donde no deberían, recursos que nunca se activan, caminos que nadie recorre.
- **Probar valores extremos.** ¿Qué pasa si no llega nadie? ¿Y si llegan todos juntos? ¿Y si el recurso tiene capacidad cero? El comportamiento en los extremos suele ser donde se esconden los errores.

- **Verificar la conservación de entidades.** Las entidades creadas deben ser iguales a las que salieron más las que quedaron adentro del sistema. Si no cierra, hay entidades perdidas o duplicadas.
- **Contrastar con un caso de solución conocida.** Un modelo sencillo cuyo resultado se puede calcular analíticamente con Teoría de Colas (Eje 5) sirve como banco de pruebas del modelo grande.
- **Revisión por otra persona.** Que alguien que no construyó el modelo lo recorra y pregunte. Es sorprendentemente efectivo.

Advertencia: la animación es una ayuda de verificación, **no una prueba**. Un modelo puede verse perfecto en pantalla, con las entidades circulando prolijamente, y tener mal parametrizada una distribución o invertida una regla de prioridad. Lo que se ve bien no está necesariamente bien.

VERIFICAR NO ES VALIDAR

Conviene fijar la distinción de una vez, porque se confunden permanentemente:



Un modelo puede estar perfectamente verificado y ser completamente inválido

	Verificación	Validación
Pregunta	¿Construimos bien el modelo?	¿Construimos el modelo correcto?
Compara	El modelo implementado contra el modelo conceptual	El modelo contra el **sistema real**
Naturaleza	Interna, lógica y de programación	Externa, estadística y de criterio experto
Ejemplo	La regla de prioridad quedó al revés en el código	El modelo asume llegadas parejas y en el sistema real hay un pico al mediodía

Un modelo puede estar perfectamente verificado y ser completamente inválido: implementa sin errores un sistema que no es el que estamos estudiando. La validación se desarrolla en el **Eje 6**, junto con el análisis de los datos de entrada y de los resultados.

SEMILLAS, RÉPLICAS Y REPRODUCIBILIDAD

Última cuestión de implementación, y es una que conecta directo con el Eje 2. Como el modelo es estocástico, **cada corrida da un resultado distinto**. Eso obliga a tres cuidados prácticos:

- **Fijar la semilla** cuando se quiere reproducir exactamente una corrida —para depurar un error, o para que otra persona obtenga los mismos números—. Un resultado que no se puede reproducir no se puede discutir.
- **Correr varias réplicas** con semillas distintas cuando se quiere estimar un indicador. Un promedio de una sola corrida no dice casi nada; cuántas réplicas hacen falta se determina con las técnicas del Eje 6.
- **Usar corrientes separadas** para cada fuente de aleatoriedad —una para las llegadas, otra para los tiempos de atención, otra para las fallas—. Además de ordenar, esto permite comparar dos escenarios sometiéndolos a «la misma suerte»: es la técnica de **números aleatorios comunes**, una de las formas de reducción de varianza que se ve en el Eje 6.

Los simuladores traen todo esto resuelto, pero hay que saber que existe y activarlo. La reproducibilidad no es un lujo académico: es lo que permite que las conclusiones de un estudio de simulación se puedan auditar.

COMPLEMENTO A — EVOLUCIÓN DE LOS LENGUAJES Y EL SOFTWARE

Este material no forma parte del núcleo de la unidad. Sirve para entender por qué las herramientas actuales son como son.

Etapa	Período	Qué caracterizó a la etapa
1. Programación artesanal	Décadas de 1950 y 1960	Modelos escritos íntegramente en FORTRAN o ALGOL. Cada proyecto reconstruía su propio motor de eventos. Costo altísimo y errores repetidos.
2. Lenguajes de simulación	1960-1980	Aparecen GPSS, SIMSCRIPT, SIMULA, GASP. El motor viene incorporado. Se consolidan las dos visiones del mundo y nace, de paso, la programación orientada a objetos.
3. Interfaces gráficas	1980-1995	La computadora personal permite construir el modelo con diagramas de bloques y animación 2D. La simulación se abre a usuarios que no programan. Surgen los paquetes comerciales que todavía existen.
4. Objetos y 3D	1995-2010	Bibliotecas de objetos reutilizables, animación tridimensional, jerarquías y modelos modulares. La animación pasa a ser también una herramienta de comunicación con quien toma la decisión.
5. Integración y análisis	2010 en adelante	Conexión directa con bases de datos y sistemas de gestión, ejecución en la nube, tableros de resultados, experimentación y optimización automáticas, plataformas multimétodo y modelos que se alimentan de datos en línea (**gemelos digitales**).

El hilo conductor es constante: **cada etapa automatizó lo que en la anterior era trabajo manual del analista**. Primero el motor de eventos, después la construcción del modelo, después la reutilización de componentes, ahora la conexión con los datos y la experimentación. Lo que ninguna etapa automatizó todavía es la parte conceptual: entender el problema, decidir qué modelar y juzgar si el resultado tiene sentido.

COMPLEMENTO B — CRITERIOS PARA SELECCIONAR UNA HERRAMIENTA

Cuando en la vida profesional haya que elegir un software de simulación, la decisión no debería tomarse por la calidad de la animación ni porque «es el que usa todo el mundo». Conviene evaluar, como mínimo:

Criterio	Qué preguntarse
Paradigma	¿El problema es de eventos discretos, continuo, de agentes o mixto? ¿La herramienta soporta ese paradigma?
Capacidad de representación	¿Permite modelar las restricciones, prioridades y reglas particulares del sistema, o hay que forzar el modelo para que entre?
Flexibilidad	¿Se puede escribir lógica propia cuando los objetos estándar no alcanzan? ¿En qué lenguaje?
Prestaciones estadísticas	¿Qué indicadores recolecta solo? ¿Permite definir los propios? ¿Calcula intervalos de confianza? ¿Exporta los datos crudos?
Experimentación	¿Automatiza réplicas, escenarios y análisis de sensibilidad? ¿Incluye optimización?
Integración	¿Se conecta con planillas, bases de datos o el sistema de gestión de la empresa?
Velocidad de ejecución	¿Corre modelos del tamaño que se necesita en un tiempo razonable?
Curva de aprendizaje	¿Cuánto tarda el equipo en ser productivo? ¿Hay documentación, ejemplos, cursos y comunidad?
Costo total	No solo la licencia: capacitación, mantenimiento, hardware y tiempo de desarrollo.
Disponibilidad académica	¿Existe una versión educativa o gratuita que permita practicar fuera del ámbito de trabajo?

Un último criterio, menos técnico pero decisivo: **la herramienta debe permitir comunicar los resultados a quien toma la decisión**. Un modelo excelente cuyos resultados nadie entiende no cambia nada.

REFERENCIAS BIBLIOGRÁFICAS

BIBLIOGRAFÍA OBLIGATORIA DE LA CÁTEDRA

- COSS BU, Raúl. Simulación: un enfoque práctico. Limusa, 1992.
- SHANNON, Robert E. Simulación de sistemas: diseño, desarrollo e implementación. Trillas, 1998.
- LAW, Averill M. Simulation Modeling and Analysis. 5.ª ed. McGraw-Hill, 2015. (Capítulos sobre mecanismos de avance del tiempo, lenguajes y selección de software de simulación.)

BIBLIOGRAFÍA COMPLEMENTARIA

- BANKS, Jerry; CARSON, John S.; NELSON, Barry L.; NICOL, David M. Discrete-Event System Simulation. 5.ª ed. Prentice Hall, 2010.
- MARMOLEJO, Isaías Simón; ORTIZ DOMÍNGUEZ, Manuel. Un primer paso a la simulación con FlexSim. 2.ª ed. Flexin Iberia, 2019.
- BEAVERSTOCK, Malcolm; GREENWOOD, Allen; NORDGREN, William. Applied Simulation: Modeling and Analysis Using FlexSim. 5.ª ed. FlexSim Software Products, 2017.
- ZEIGLER, Bernard P.; MUZY, Alexandre; KOFMAN, Ernesto. Theory of Modeling and Simulation. 3.ª ed. Academic Press, 2018. (Formalismo DEVS.)

DOCUMENTACIÓN DE HERRAMIENTAS

- Documentación oficial de SimPy — SimPy: Discrete event simulation for Python.
- Manuales y materiales de capacitación de FlexSim, Arena, Simio, AnyLogic y Vensim.